PRACTICAL 1

AIM: Project Definition and objective of the specified module and Perform Requirement Engineering Process for a **QR-Based Dine-In Ordering Platform**.

Introduction

Manual dine-in service workflows in restaurants and hotel dining halls often introduce operational bottlenecks, including delayed order-taking, mis-heard or mis-relayed items, and slow bill settlement. The **QR-Based Dine-In Ordering Platform, Dinora,** addresses these bottlenecks by serving as a browser-based table-ordering ecosystem built to remove the waits guests experience between sitting down and being served. By centralizing menu browsing, live table sessions, and order-status tracking, this application optimizes floor throughput for staff while granting guests frictionless self-service ordering and transparent order tracking

Objectives

Primary Objective

- Develop a robust, browser-based web application platform to automate, record, and optimize real-time table ordering, live order tracking, and payment settlement for dine-in restaurants and hotels.

Secondary Objectives

- Automate the order-taking pipeline across the guest's visit to systematically capture table orders alongside real-time menu availability.
- Minimize human error dependencies by removing verbal order relay between guest, server, and kitchen counter.
- Implement an intuitive self-service ordering flow to optimize guest journeys via a QR-triggered menu and transparent, live order status tracking.
- Provide restaurant owners with a real-time counter dashboard of incoming orders, sorted by table, to optimize floor staffing and service speed.

Project Definition

Scope

Dinora delivers a comprehensive web application designed to govern the dine-in ordering lifecycle through a QR-triggered, session-based architecture. The scope encompasses:

- **Live Table Session Architecture**: Provisioning a QR-triggered, session-based ordering flow — opening straight in the guest's browser with no login or install — that keeps every order for a table tied to one running session from first order to final payment.
- **Menu & Order Engine**: Maintaining a browsable digital menu with photos, dietary tags, and descriptions, while running deterministic logic to handle order placement, add-on requests, and instant availability updates.
- **Counter Dashboard Framework**: Facilitating a real-time counter queue that surfaces incoming orders tagged with table number and items, moving each ticket through New, Preparing, Served, and Paid & Closed states.
- **Owner & Payment Systems**: Structuring settlement routines to close out a table's session via cash, UPI, or card, and to update menu items and prices live across every table.

Modules

1. **QR Table Menu Module**: Governs the QR scan-to-menu flow, opening the full menu instantly in any phone browser with no app download or login required.
2. **Live Table Session Module**: Implements the shared ordering session for a table, keeping every order — including later rounds — tied to one running tab until payment.
3. **Multi-Guest Ordering Module**: Controls simultaneous ordering from multiple guests at the same table, letting everyone add to the order from their own phone.
4. **Order Ticket & Status Module**: Manages ticket creation at the counter, handles order-status transitions (New, Preparing, Served), and shifts records across the session lifecycle.
5. **Add-on & Availability Module**: Handles special requests noted directly on the ticket (extra items, no onion, less spicy) and lets staff mark dishes unavailable the instant they run out.
6. **Counter Dashboard Module**: Executes real-time queue monitoring for incoming orders sorted by table, and flags tables that need attention.
7. **Payment & Session Closure Module**: Collects cash, UPI, or card settlement and closes out the table's session once payment is confirmed.

Requirement Engineering Process

Stakeholder Identification

- **Primary Stakeholders**: Restaurant and hotel floor staff (who monitor the counter dashboard and serve incoming orders), Guests (who scan, browse, order, and pay from the table), and Restaurant Owners (responsible for menu updates and floor staffing decisions).
- **Secondary Stakeholders**: Hotel & Restaurant Management (who review service-time and turnover trends) and Platform Developers (responsible for application maintenance and system stability).

Requirement Elicitation

- **Interviews**: Structured interviews are conducted with restaurant floor staff to map out precise manual pain points regarding order relay, mis-heard items, and bill settlement delays.
- **Surveys**: Digital questionnaires are distributed to regular dine-in guests to capture menu-browsing preferences, ordering-flow demands, and waiting-time friction zones.
- **Observation**: Direct observation of a busy dining floor's order-taking and billing cycle is executed to identify systemic bottlenecks in the current waiter-dependent workflow.

Requirement Analysis

- **Functional Requirements**:
    - The system must open a live table session the instant a guest scans the table's QR code, with no login or install required.
    - The system must load the full menu with photos, dietary tags, and descriptions, and reflect availability changes instantly.
    - The system must allow multiple guests at the same table to add to a single shared order from their own phones.
    - The system must create an order ticket that reaches the counter dashboard instantly, tagged with table number and requested items.
    - The system must track order status through New, Preparing, Served, and Paid & Closed states, visible to both guest and counter.

- **Non-Functional Requirements**:
    - **Performance**: The system must achieve rapid execution, with the menu and order ticket loading in the guest's browser without a noticeable delay.
    - **Scalability**: The application architecture must easily sustain concurrent table sessions across a full dining floor during peak hours.
    - **Reliability**: The system must keep a table's session accurately tied to that table until payment closes it out, with no order loss on repeat scans.
    - **Availability**: The platform infrastructure must operate continuously through service hours, so a guest is never left unable to order.

Requirement Specification

- **Documentation**: An official Software Requirements Specification (SRS) document is structured to capture all functional and non-functional engineering limits.
- **Use Case Diagrams**: Unified Modeling Language (UML) use case diagrams are developed to map guest, counter-staff, and owner workflows across the ordering session.
- **User Stories**: Granular agile user stories are written to trace guest goals — scan, browse, order, track, and pay — across each screen of the ordering flow.

Requirement Validation

- **Reviews**: Regular technical walk-throughs and review meetings are held with restaurant stakeholders to check requirement clarity and structural completeness.
- **Prototyping**: Interactive wireframes and rapid UI mockups of the QR menu and counter dashboard are built to validate ordering-flow interactions with floor teams.
- **Testing**: Comprehensive requirements trace testing is executed to confirm that every planned ordering-flow feature corresponds directly to an engineering deliverable.

Lab Activities

Activity 1: Project Definition

- **Task**: Articulate the scope, objectives, and domain boundaries of the QR-based dine-in ordering platform, Dinora.
- **Deliverable**: A comprehensive project definition charter detailing operational limits and target user ecosystems (hotels and fine casual dining).

Activity 2: Stakeholder Identification

- **Task**: Catalog and group all individuals interacting with the dine-in ordering environment to define access needs.
- **Deliverable**: A stakeholder analysis matrix defining guest, floor-staff, and owner access needs.

Activity 3: Requirement Elicitation

- **Task**: Execute dynamic stakeholder interviews, surveys, and workflow observations on an operational dining floor.
- **Deliverable**: A raw requirement elicitation log capturing ordering-flow pain points, structural needs, and session conditions.

Activity 4: Requirement Analysis

- **Task**: Dissect, categorize, and prioritize gathered ordering-flow features into explicit functional and non-functional requirements.
- **Deliverable**: An analytical requirement categorization report detailing dependencies, parameters, and constraints.

Activity 5: Requirement Specification

- **Task**: Convert analyzed requirement statements into formal engineering models by authoring an SRS document accompanied by use case diagrams.
- **Deliverable**: A formally structured, baseline Software Requirements Specification (SRS) document.

Activity 6: Requirement Validation

- **Task**: Validate the specified boundaries with primary stakeholders using UI prototypes, structural reviews, and interface test cases.
- **Deliverable**: A signed requirement validation audit report confirming alignment across guest and floor-staff needs.

Conclusion

This initial engineering phase establishes a structured approach to scoping, defining, and formalizing the engineering foundation for Dinora. By systematically translating practical dine-in ordering workflows, live table-session states, and payment policies into verifiable technical requirements, a clear blueprint is created for subsequent software layers. Following

these steps ensures the application remains securely anchored to stakeholder needs, minimizing development deviations during architecture design, session-state design, and final interface implementation.

PRACTICAL 2

AIM: Identify Suitable Design and Implementation models from the different software engineering models for a **QR-Based Dine-In Ordering Platform (Dinora)**.

Objectives

- Understand the explicit structural, functional, and operational requirements governing the Dinora ordering platform.
- Explore and analyze diverse software development lifecycles (including Waterfall, Agile, Spiral, and the V-Model) to ascertain their individual trade-offs.
- Identify and justify the selection of the most adaptive, scalable software engineering model specifically for implementing a QR-triggered, session-based dine-in ordering web application.
- Architect a structural prototype blueprint covering use case components, live-session data models, and counter-dashboard interface boundaries.
- Establish a systematic testing and evaluation plan to verify implementation functionality against core ordering-flow rules.

Prerequisites

- Software Engineering Principles: Foundational understanding of the Software Development Lifecycle (SDLC), iterative release methods, and agile framework mechanics.
- Front-End Web Architecture: Core competency in browser-based session handling, responsive UI layout, and mobile-first interaction design.
- Data & Session Management: Practical experience with structuring live order data, table-session state, and real-time status updates.
- Tooling Proficiency: Familiarity with standard design and prototyping tools, version control protocols via Git, and UI wireframing clients.

Lab Setup

- **Software Environment**:
    - Delivery Format: Browser-based web app, QR-triggered, no install required (native app planned next).

- Primary Deployment: acenetstudios.vercel.app hosting for the current web app prototype.
- Interface Scope: QR table menu, live table sessions, and a counter dashboard for incoming orders.
- Target Venues: Hotels, fine casual dining, and quick-casual dining rooms.
- Payment Channels: Cash, UPI, and card, closed out within the same table session.

- **Hardware Specification**: Standard development computer station for the web app, alongside a counter-side device (tablet or desktop) to run the dashboard, and any guest smartphone with a camera and browser to scan and order.
- **Data Context**: Sample menu data simulating dish entries, dietary tags, table sessions, and order-status transitions across New, Preparing, Served, and Paid & Closed.

Activity 1: Requirement Analysis

- **Objective**: Define the boundaries of the system by transforming the raw service rules of a dine-in floor into rigid functional items.
- **Tasks**:
  - Detail the system's core purposes, prioritizing QR-triggered session start, menu browsing, order placement, and payment settlement.
  - Enumerate screen-specific components, tracking the guest ordering flow, the counter dashboard, and status-tracking views.
  - Group stakeholders into access tiers to map out guest, floor-staff, and owner privileges.
- **Outcome**: A detailed, verified requirement specification document aligned with dine-in service workflows.

Activity 2: Exploration of Software Engineering Models

- **Objective**: Analyze the structural behavior of industry-standard development paradigms against the specific demands of the project.
- **Tasks**:
  - Study the core mechanics of the sequential Waterfall process, iterative Agile sprints, risk-driven Spiral strategies, and verification-heavy V-Models.

- Evaluate the advantages and disadvantages of each individual framework when applied to web application patterns.
- Grade every paradigm based on variable parameters such as timeline flexibility, client input iterations, and architectural complexity.

- **Outcome**: A detailed comparative evaluation matrix mapping out the capabilities of software engineering lifecycles.

Software Engineering Model	Pros (Context of Dinora)	Cons (Context of Dinora)	Flexibility	Stakeholder Input
Waterfall	Clear initial structures; straightforward documentation maps.	High failure vulnerability; unable to handle changes mid-stream.	Rigid / Low.	Only at start / end.
Agile (Recommended)	Iterative feature delivery; seamless integration of future enhancements (e.g., native app, owner analytics).	Demands constant team communication; requires rigid sprint discipline.	Highly Adaptive.	Continuous per iteration.
Spiral	Deep risk analysis; excellent for intricate payment-settlement modules.	Highly expensive execution; over-complicated for mid-sized web stacks.	Moderate.	At every phase review.
V-Model	Strict validation links; test scenarios map directly to requirements.	Lacks early prototyping paths; making alterations late is highly expensive.	Rigid.	Only during alignment.

Activity 3: Selection of the Suitable Model

- **Objective**: Formally declare and defend the optimum execution framework to guide the system's development lifecycle.
- **Tasks**:
  - Justify utilizing an **Agile (Scrum/Kanban)** framework by highlighting the phased nature of the application (web app now, native app next, owner analytics later).
  - Map out the integration path for rolling out incremental phases, ensuring the core QR-ordering flow releases early while advanced extensions launch later.

- Plan feedback check loops at the conclusion of every feature delivery step to verify UI layout and ordering-flow controls.

- **Outcome**: An official model selection report containing engineering justifications for the selected lifecycle.

Activity 4: System Design

- **Objective**: Construct the structural, data, and behavioral layout blueprints defining the platform's guest and counter-side environments.
- **Tasks**:
  - Create comprehensive Use Case diagrams separating the roles of Guests and Counter Staff against core actions like ordering or closing a session.
  - Design session and order data models specifying how a table's live session links its orders, statuses, and payment state.
  - Draft high-level data handling flows mapping a guest's QR scan through menu load, order placement, and counter-ticket delivery.
- **Outcome**: System design documents, session-state maps, and logical system flowcharts.

Activity 5: Implementation

- **Objective**: Translate architectural design diagrams into executable, production-ready front-end software components.
- **Tasks**:
  - Initialize the workspace environment, setting up the browser-based ordering flow alongside the counter dashboard UI scaffolding.
  - Program core session-handling logic by defining state models for tables, orders, and payment status.
  - Integrate functional UI components including the QR-triggered menu, order-status tracker, and counter ticket queue.
- **Outcome**: A fully functional, responsive web app prototype of Dinora.

Activity 6: Testing and Evaluation

- **Objective**: Verify system performance metrics and check for ordering-flow discrepancies across session states.

- **Tasks**:
    - Execute strict testing over order-ticket logic, verifying that a second round of orders lands on the same running session rather than a fresh one.
    - Conduct verification on session handling to confirm a closed table cannot silently receive new orders after payment.
    - Measure baseline performance to confirm the menu and order flow load quickly on a guest's phone browser.
- **Outcome**: Comprehensive verification test reports detailing system performance metrics and ordering-flow compliance validation.

Activity 7: Documentation and Reporting

- **Objective**: Package engineering files, lifecycle choices, and testing summaries into a structured practical lab portfolio.
- **Tasks**:
    - Author a detailed technical summary document detailing development execution metrics and system build achievements.
    - Embed structural engineering schematics, session-state examples, and UI screen layout captures.
    - Compile an analytical retrospective breaking down encountered session-handling errors, environment configurations, and reliability optimization metrics.
- **Outcome**: A complete, peer-reviewed engineering lab report portfolio.

Expected Outcomes

Upon successful completion of this practical, students will achieve the following deliverables:

- **SDLC Evaluation Matrix**: A comprehensive comparative document that evaluates traditional and modern software engineering lifecycles against the architecture of a real-time ordering web application.
- **Model Selection Justification Report**: A formalized defense detailing why the Agile framework aligns best with the phased rollout of the system (web app, native app, owner analytics).

- **Functional System Architecture Blueprint**: A conceptual architectural layout mapping out the guest-to-counter request pipeline from QR scan down through order-ticket delivery.
- **Ordering Flow Mockups**: High-fidelity wireframe interfaces displaying the QR menu, live order tracking, and the counter dashboard built using responsive layouts.

Assessment Criteria

The practical submission will be graded rigorously based on the following specific engineering indicators:

Assessment Component	Weight	Grading Parameters & Quality Indicators
Requirement Analysis & Scope	**20%**	Clear classification of functional user journeys (Guest vs. Counter-Staff workflows) and realistic scoping of non-functional operational boundaries.
Lifecycle Evaluation Quality	**25%**	Analytical depth in contrasting structural patterns (Waterfall, Spiral, V-Model, Agile) and the accuracy of matching Agile sprints to iterative release phases.
System Blueprint Architecture	**25%**	Technical accuracy of conceptual data flow mappings, live-session state representations, and counter-dashboard routing steps.
Prototype Presentation & Design	**15%**	Logical arrangement of UI control structures, responsive design consistency, and clean separation of guest and counter layout states.
Technical Report & Documentation	**15%**	Professional writing style, structured formatting according to lab manual regulations, and precise usage of core ordering-platform engineering terminology.

PRACTICAL 3

AIM

Prepare Software Requirement Specification (SRS) for the selected module.

Objectives

1. Understand the functional and non-functional requirements of the system.
2. Identify the key modules and their interactions.
3. Document the requirements in a structured SRS format.
4. Ensure the SRS aligns with the goals of Dinora, the QR-based dine-in ordering platform.

Prerequisites

1. Basic understanding of dine-in restaurant service flow and its significance in guest experience.
2. Familiarity with Software Requirement Specification (SRS) documentation.
3. Knowledge of browser-based session handling and its application in ordering systems.

Tools and Resources

1. Software Tools: Microsoft Word, Google Docs, or any SRS template tool.
2. Reference Materials: IEEE SRS template, dine-in ordering case studies, and QR-ordering UX literature.
3. Hardware: Computer with internet access.

Lab Activity

Step 1: Understand the System

1. Overview: Dinora is designed to let guests scan a QR code at their table, browse the menu, and order for everyone in one shared session. It integrates a live counter dashboard to route orders instantly.
2. Key Modules:
    - **QR Table Menu Module:** Collects orders from guests via the QR-triggered menu.
    - **Live Table Session Module:** Keeps every order for a table tied to one running session.
    - **Counter Dashboard Module:** Surfaces incoming tickets to the counter, sorted by table.

- **Order Status Module:** Provides a dashboard for guests and counter staff.
- **Payment Module:** Closes out a table session on cash, UPI, or card settlement.

Step 2: Select a Module

Choose one module from the system (e.g., Live Table Session Module) for which you will prepare the SRS.

Step 3: Gather Requirements

1. Functional Requirements:
   - Define the inputs, processes, and outputs of the selected module.
   - Example for Live Table Session Module:
     - **Input:** QR scan event and table identifier.
     - **Process:** Open a session, attach subsequent orders to the same session until payment.
     - **Output:** A running order ticket tied to the table, visible to guest and counter.
2. Non-Functional Requirements:
   - **Performance:** The menu and session should load in the guest's browser without a noticeable delay.
   - **Scalability:** The system should handle concurrent table sessions across a full dining floor.
   - **Security:** Ensure order and payment data privacy and reliable session handling.
3. User Requirements:
   - Define the needs of end-users (guests, counter staff, owners).
   - **Example:** Counter staff should receive new orders in an easy-to-read ticket format.

Step 4: Document the SRS

Use the IEEE SRS template or a similar structure to document the requirements. Include the following sections:

4. Introduction:

- Purpose of the SRS.
- Scope of the selected module.
- Definitions and acronyms.

5. Overall Description:
   - Module functionality.
   - User characteristics.
   - Constraints and assumptions.
6. Specific Requirements:
   - Functional requirements.
   - Non-functional requirements.
   - Interface requirements.
7. Appendices:
   - Glossary, references, and additional information.

Step 5: Review and Validate

8. Cross-check the SRS with the system objectives.
9. Ensure clarity, completeness, and consistency.
10. Share the SRS with peers or instructors for feedback.

Expected Outcome

A well-structured Software Requirement Specification (SRS) document for the selected Dinora module.

PRACTICAL 4

AIM

Develop Software project management planning (SPMP) for the specified module.

Objectives

1. Understand the requirements of Dinora, the QR-based dine-in ordering platform.
2. Define the scope, deliverables, and milestones of the project.
3. Identify resources, roles, and responsibilities for the project.
4. Develop a project schedule and timeline.
5. Plan risk management, quality assurance, and communication strategies.
6. Create a comprehensive SPMP document.

Prerequisites

1. Basic understanding of software project management.
2. Familiarity with browser-based ordering flows and dine-in service operations.
3. Knowledge of tools like Trello, Jira, or Asana for project planning.
4. Understanding of QR-triggered, session-based ordering concepts.

Tools and Resources

1. Project management tools: Trello, Jira, Asana, or Microsoft Project.
2. Documentation tools: Microsoft Word, Google Docs, or LaTeX.
3. Front-end frameworks: React or similar, for reference.
4. Sample SPMP templates.

Lab Procedure

Step 1: Project Overview

1. Define the Problem Statement:
    - Develop a QR-based dine-in ordering platform to remove the waits guests experience between sitting down and being served.
    - The system should include features like QR menu access, live table sessions, counter dashboard, and flexible payments.
2. Scope of the Project:
    - **Identify the modules:** QR Table Menu, Live Table Session, Counter Dashboard, Order Status Tracking, Payments.

- Define the boundaries of the project (web app now, native app next).

3. Stakeholders:
   - **Identify stakeholders:** Guests, floor staff, restaurant owners, and developers.

Step 2: Project Deliverables and Milestones

4. Deliverables:
   - SPMP document.
   - QR-triggered menu and ordering flow.
   - Counter dashboard prototype.
   - Final deployed web app.
5. Milestones:
   - Requirement gathering and analysis (Week 1-2).
   - Design and prototyping (Week 3-4).
   - Ordering flow and session-handling development (Week 5-6).
   - Integration and testing (Week 7-8).
   - Deployment and documentation (Week 9-10).

Step 3: Resource Planning

6. Human Resources:
   - **Project Manager:** Oversees the project.
   - **Front-End Developer:** Builds the ordering flow and counter dashboard.
   - **UI/UX Designer:** Designs the guest and counter interfaces.
   - **QA Engineer:** Ensures quality and testing.
   - **Restaurant Consultant:** Provides domain expertise on floor operations.
7. Hardware and Software Resources:
   - Computers with front-end frameworks installed.
   - Cloud hosting for deployment (e.g., Vercel).
   - Mobile devices for testing the QR-scan flow.

Step 4: Project Schedule

8. Use a project management tool to create a Gantt chart.
9. Define tasks and subtasks for each milestone.
10. Allocate time and resources for each task.
11. Identify dependencies between tasks.

Step 5: Risk Management

12. Identify Risks:
    - Guest data and payment privacy concerns.
    - Delays in counter-dashboard integration.
    - Session-handling issues across simultaneous table orders.
13. Mitigation Strategies:
    - Implement secure payment handling and data protection practices.
    - Allocate buffer time in the schedule.
    - Conduct regular integration testing across concurrent sessions.

Step 6: Quality Assurance

14. Define quality standards for each deliverable.
15. Plan testing phases: Unit testing, integration testing, and user acceptance testing.
16. Assign QA tasks to the QA engineer.

Step 7: Communication Plan

1. Define communication channels (e.g., email, Slack, meetings).
2. Schedule regular team meetings and progress reviews.
3. Assign a communication coordinator.

Step 8: Documentation

1. Create the SPMP document with the following sections:
    - Introduction (Purpose, Scope, Objectives).
    - Project Organization (Team Structure, Roles).
    - Managerial Process (Schedule, Risk Management, Quality Assurance).
    - Technical Process (Tools, Frameworks, Development Methodology).

- Appendices (Glossary, References).

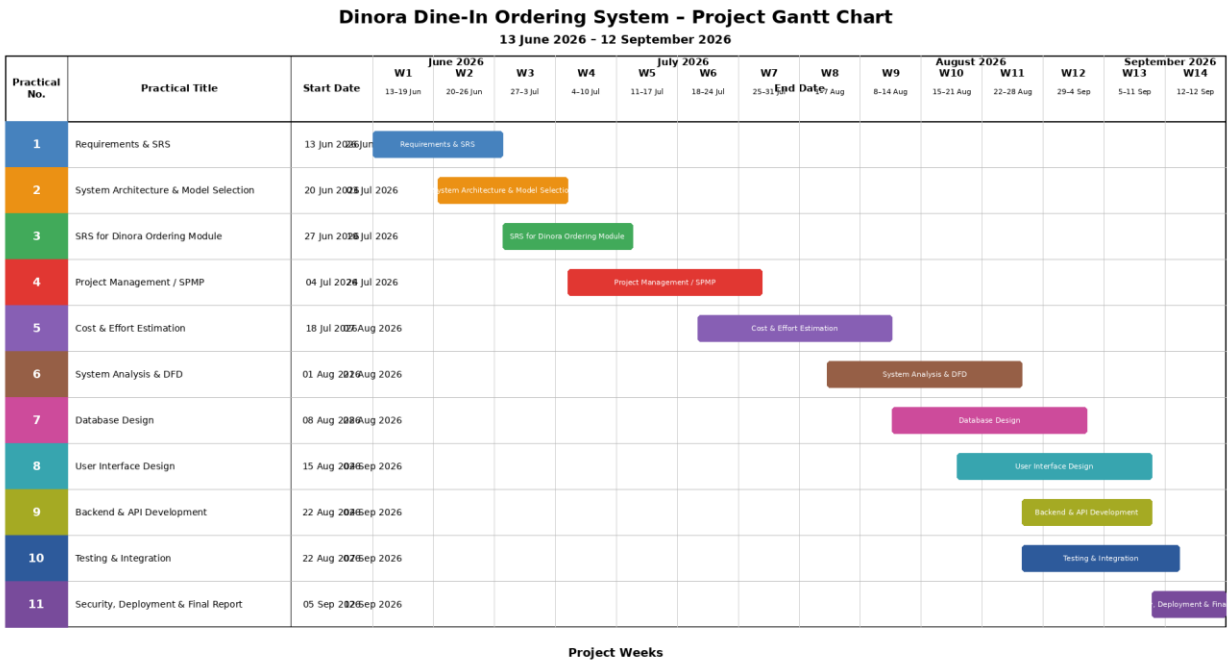
Expected Outcome

1. A comprehensive SPMP document for Dinora.
2. A clear project roadmap with defined milestones, deliverables, and timelines.
3. A risk management and quality assurance plan.

Assessment

1. Evaluate the completeness and clarity of the SPMP document.
2. Assess the feasibility of the project schedule and resource allocation.
3. Review the risk management and quality assurance strategies.

Gantt Chart:



PRACTICAL 5

AIM

Do Cost and Effort Estimation using different Software Cost Estimation models.

Objectives

- Understand the requirements of Dinora, the QR-based dine-in ordering platform.
- Apply software cost estimation models (e.g., COCOMO, Function Point Analysis, and Expert Judgment) to estimate development costs and effort.
- Compare and analyze the results from different estimation models.

Prerequisites

- Basic understanding of software engineering principles.
- Familiarity with cost estimation models (e.g., COCOMO, Function Point Analysis).
- Knowledge of browser-based session and ordering-flow concepts.
- Tools: Spreadsheet software (e.g., Excel, Google Sheets), COCOMO calculators, and Function Point Analysis tools.

Requirements for Dinora

The system will include the following features:

QR Table Menu Module:

- QR-triggered menu opening instantly in the browser.
- Photos, dietary tags, and descriptions for each dish.

Live Table Session Module:

- Session-based order tracking tied to a table.
- Multi-guest ordering from the same table.

Counter Dashboard:

- Real-time queue of incoming orders sorted by table.
- Alerts for tables needing attention.

Backend:

- Session store for tracking table state, orders, and status.
- APIs for integration with the counter dashboard and payment flow.

Security:

- Secure handling of payment data and session identifiers.

Software Cost Estimation Models

COCOMO (Constructive Cost Model)

- Steps:
    - Classify the project as Organic, Semi-Detached, or Embedded.
    - Estimate the size of the software in KLOC (Kilo Lines of Code).
    - Use the COCOMO formula:
        - Effort (E) = a * (KLOC)^b
        - Development Time (D) = c * (E)^d
        - Cost = Effort * Cost per Person-Month
    - Adjust for cost drivers (e.g., complexity, team experience).
- Example:
    - KLOC = 12
    - Project Type: Organic
    - Effort (E) = 2.4 * (12)^1.05 = 32 person-months
    - Development Time (D) = 2.5 * (32)^0.38 = 9 months
    - Cost = 32 * 10,000 = 320,000

Function Point Analysis (FPA)

- Steps:
    - Identify and count functional components (e.g., inputs, outputs, inquiries, files, interfaces).
    - Assign complexity weights to each component.
    - Calculate Unadjusted Function Points (UFP).

- Apply Value Adjustment Factor (VAF) based on system characteristics.
- Convert Function Points to effort and cost using historical data.

- Example:
  - UFP = 120
  - VAF = 1.1
  - Adjusted FP = 120 * 1.1 = 132
  - Effort = 132 * 20 hours/FP = 2,640 hours
  - Cost = 2,640 * 50/hour = 132,000

Expert Judgment

- Steps:
  - Consult with software development experts.
  - Provide them with system requirements and features.
  - Collect estimates for effort, time, and cost.
  - Average the estimates to get a final value.
- Example:
  - Expert 1: 300,000, 8 months
  - Expert 2: 340,000, 10 months
  - Average: 320,000, 9 months

Lab Tasks

Task 1: Define System Requirements

- List all modules and features of Dinora.
- Estimate the size of the software in KLOC or Function Points.

Task 2: Apply COCOMO Model

- Use the COCOMO model to estimate effort, time, and cost.
- Adjust for cost drivers specific to the project.

Task 3: Apply Function Point Analysis

- Count functional components and calculate Adjusted Function Points.
- Convert Function Points to effort and cost.

Task 4: Use Expert Judgement

- Consult with peers or instructors to get expert estimates.
- Compare these estimates with the results from COCOMO and FPA.

Task 5: Compare and Analyze Results

- Create a table comparing effort, time, and cost estimates from all three models.
- Discuss the strengths and limitations of each model.

Expected Results

- A detailed cost and effort estimation report for Dinora.
- A comparison table showing results from COCOMO, FPA, and Expert Judgment.
- Insights into which model is most suitable for this project.

Conclusion

This lab provides hands-on experience in estimating the cost and effort required to develop Dinora. By applying different software cost estimation models, students will gain insights into the challenges and considerations involved in software project planning.

PRACTICAL 6

AIM

Prepare System Analysis and System Design of identified Requirement specifications using structure design as DFD with data dictionary and Structure chart for the specific module.

Objectives

1. Understand the requirements of Dinora, the QR-based dine-in ordering platform.
2. Perform system analysis to identify functional and non-functional requirements.
3. Design the system using structured design techniques:
    - Create Data Flow Diagrams (DFD) for the system.
    - Develop a Data Dictionary to define data elements.
    - Design a Structure Chart for the specific module.
4. Document the system design for implementation.

Prerequisites

5. Basic understanding of system analysis and design.
6. Familiarity with structured design techniques (DFD, Data Dictionary, Structure Chart).
7. Knowledge of session-based ordering concepts and their application in dine-in systems.
8. Tools: Diagramming tools (e.g., Lucidchart, Draw.io, Visio) or pen and paper.

Lab Setup

- **Software:** Diagramming tools for creating DFDs, Data Dictionary, and Structure Charts.
- **Hardware:** Computer with internet access.
- **Reference Material:** Requirement specification document for Dinora.

Lab Tasks

Task 1: System Analysis

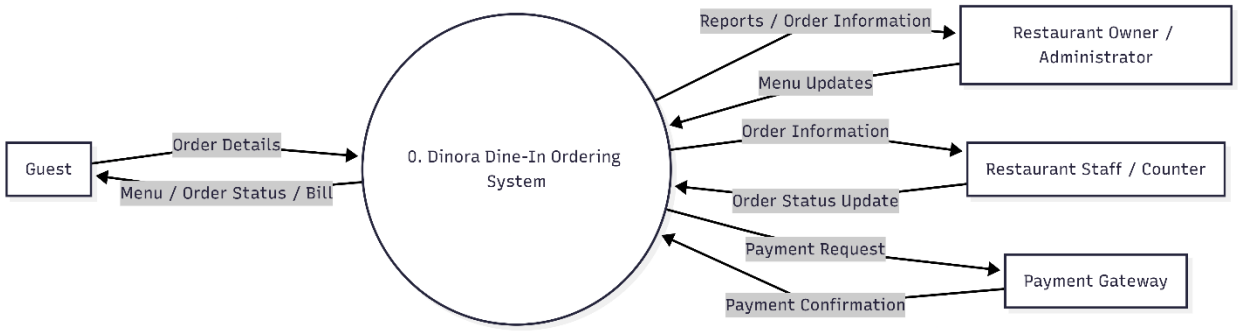
1. Requirement Gathering:
    - Identify the key stakeholders (e.g., guests, counter staff, restaurant owners).
    - Gather functional and non-functional requirements for the system.
    - Example:
        - **Functional:** Open a table session, place orders, track order status.

- **Non-functional:** Secure payment handling, real-time processing, user-friendly interface.

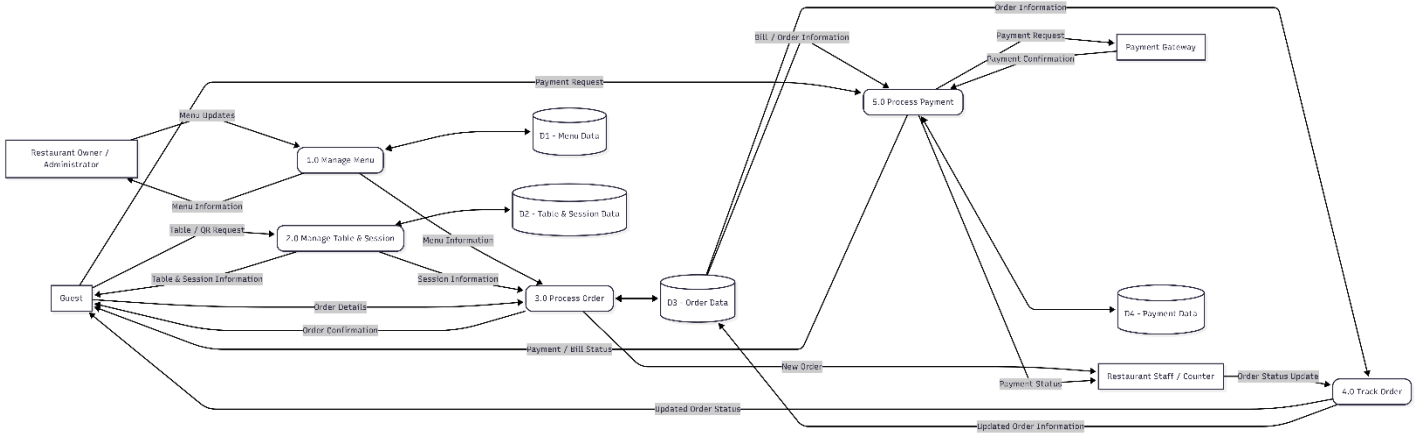
2. Requirement Specification:
   - Document the requirements in a structured format.
   - Example:
     - **Input:** QR scan, table number, dish selections.
     - **Process:** Session creation, order-ticket routing.
     - **Output:** Live order status, counter ticket, payment confirmation.

Task 2: System Design Using Structured Design Techniques

3. Data Flow Diagram (DFD):
   - Create a Level 0 (Context Diagram):
     - Represent the system as a single process interacting with external entities (e.g., Guest, Counter Staff).



   - Create Level 1 DFD:
     - Break down the system into major processes (e.g., Open Session, Place Order, Generate Ticket).



11. Data Dictionary:

- Define all data elements used in the DFD.
- Example:

Data Elements (Fields)

Data Element	Description	Data Type	Format/Range	Example
Table_ID	Unique identifier for a table	String	T-XXXX (e.g., T-0412)	T-0412
Session_ID	Unique identifier for a table session	String	S-XXXX	S-1001
Dish_ID	Unique identifier for a menu item	String	D-XXXX	D-2210
Dish_Name	Name of the menu item	String	Alphabetic	"Aloo Paratha"
Quantity	Number of a dish ordered	Integer	1-50	2
Order_Timestamp	Date and time the order was placed	DateTime	YYYY-MM-DD HH:MM:SS	2026-07-14 19:20:00
Order_Status	Current status of the order	String	"New", "Preparing", "Served", "Paid & Closed"	"Preparing"
Add_On_Note	Special request noted on the ticket	String	Free text	"Extra roti, less spicy"

Payment_Method	Method used to settle the session	String	"Cash", "UPI", "Card"	"UPI"
Session_Total	Running total for the table session	Decimal	0.00-99999.99	845.00

Data Stores (Databases/Tables)

Data Store	Description	Key Fields
Table_Sessions_DB	Stores live and closed table sessions	Session_ID, Table_ID, Order_Status, Session_Total
Menu_DB	Stores menu items, prices, and availability	Dish_ID, Dish_Name, Price, Availability
Orders_DB	Records individual order line items per session	Session_ID, Dish_ID, Quantity, Add_On_Note
Payments_DB	Contains settlement records for closed sessions	Session_ID, Payment_Method, Session_Total

Data Flows (Inputs/Outputs)

Data Flow	Source → Destination	Description
QR_Scan_Input	Guest → System	Table identifier captured when the guest scans the QR code
Order_Submission	Guest → Table_Sessions_DB	Dish selections and add-on notes for the session
Order_Ticket	System → Counter Dashboard	New order routed to the counter, tagged with table number
Status_Update	Counter Dashboard → Guest	Order status change (Preparing, Served)
Payment_Confirmation	Guest → Payments_DB	Cash, UPI, or card settlement closing the session

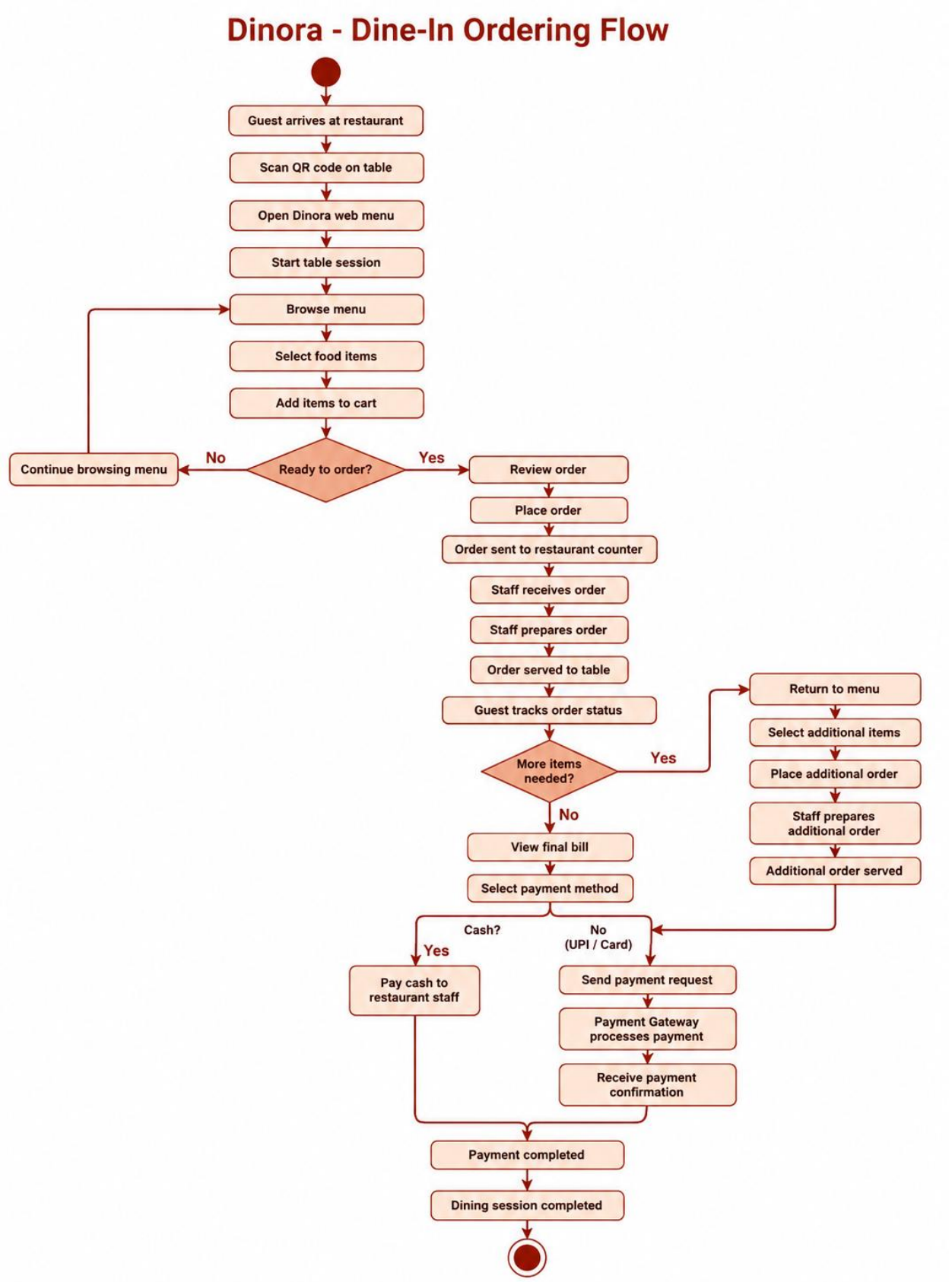
4. Structure Chart:
    - Legend:
        - **Rectangle:** Module (function/subroutine).
        - **Arrow with circle:** Data flow (input/output).

- **Arrow without circle:** Control flow (execution order).

Level 0: Root Module (Main System)

Module: Dinora_Ordering_System

- **Function:** Coordinates all system operations.
- Submodules:
  - QR_Session_Handler
  - Order_Capture
  - Counter_Routing
  - Payment_Settlement


Dinora - Dine-In Ordering Flow
Guest arrives at restaurant
Scan QR code on table
Open Dinora web menu
Start table session
Browse menu
Select food items
Add items to cart
Ready to order?
No
Continue browsing menu
Yes
Review order
Place order
Order sent to restaurant counter
Staff receives order
Staff prepares order
Order served to table
Guest tracks order status
More items needed?
Yes
Return to menu
Select additional items
Place additional order
Staff prepares additional order
Additional order served
No
View final bill
Select payment method
Cash?
Yes
Pay cash to restaurant staff
No (UPI / Card)
Send payment request
Payment Gateway processes payment
Receive payment confirmation
Payment completed
Dining session completed

Level 1: Submodule Breakdown

1. QR_Session_Handler

- **Function:** Opens or resumes a table's live session on QR scan.
- Submodules:
    - Validate_Table_Code
    - Create_Or_Resume_Session

2. Order_Capture

- **Function:** Records dish selections and add-on notes for a session.
- Submodules:
    - Load_Menu
    - Submit_Order_Items

3. Counter_Routing

- **Function:** Delivers new tickets to the counter dashboard and tracks status.
- Submodules:
    - Create_Ticket
    - Update_Order_Status
    - Notify_Guest

4. Payment_Settlement

- **Function:** Closes out the table session on successful payment.
- Submodules:
    - Collect_Payment
    - Close_Session

Key Data Flows

Module	Input Data	Output Data
QR_Session_Handler	Table_ID from QR scan	Session_ID (new or resumed)
Order_Capture	Dish_ID, Quantity, Add_On_Note	Order line items tied to Session_ID
Counter_Routing	Order line items	Order_Ticket, Order_Status
Payment_Settlement	Payment_Method, Session_Total	Session closure confirmation

Task 3: Documentation

12. Compile the system analysis and design into a comprehensive document.
13. Include:
    - Requirement specification.
    - DFDs (Level 0, Level 1, Level 2).
    - Data Dictionary.
    - Structure Chart.
14. Provide a brief explanation of each component.

Expected Output

15. System Analysis Document:
    - Clear and concise requirement specification.
16. System Design Artifacts:
    - DFDs (Level 0, Level 1, Level 2).
    - Data Dictionary.
    - Structure Chart for the specific module.
17. Lab Report:
    - Summary of the tasks performed.
    - Screenshots or images of the diagrams.
    - Explanation of the design decisions.

PRACTICAL 7

AIM

Designing the module using Object Oriented approach including a Use case Diagram with scenarios, Class Diagram and State Diagram, Collaboration Diagram, Sequence Diagram, and Activity Diagram.

Objectives

18. Understand the requirements of Dinora, the QR-based dine-in ordering platform.
19. Apply Object-Oriented Programming (OOP) principles to design the system
20. Create UML diagrams to represent the system's structure and behavior.
21. Model the live table session lifecycle from QR scan through payment.

Prerequisites

22. Basic knowledge of Object-Oriented Programming (OOP) concepts.
23. Familiarity with UML diagrams.
24. Understanding of session-based ordering concepts and their application in dine-in systems.
25. Tools: UML diagramming tools (e.g., Lucidchart, Draw.io, StarUML, or any preferred tool).

Lab Setup

26. Software: UML diagramming tool, a front-end framework (e.g., React) for reference, and any IDE for coding.
27. Hardware: Computer with internet access.
28. References: UML documentation, dine-in ordering UX case studies, and QR-session design guidelines.

Lab Tasks

Task 1: Requirements Analysis

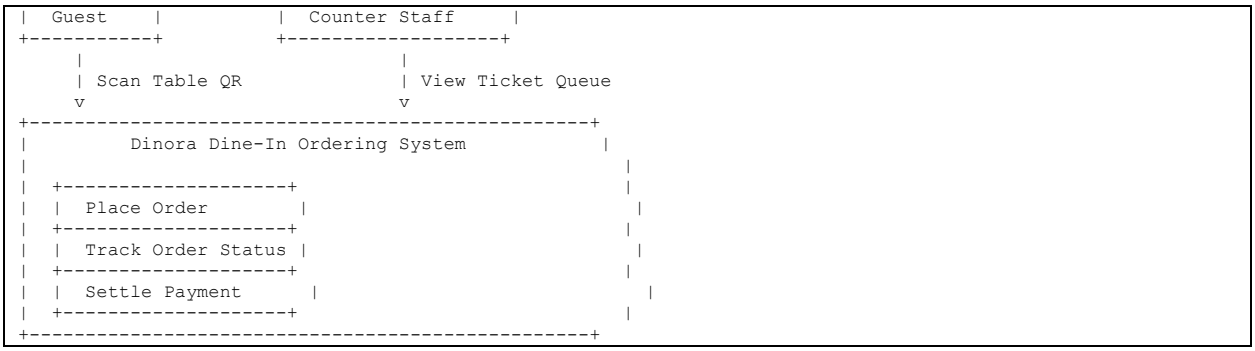
29. Objective: Identify the functional and non-functional requirements of Dinora.
30. Steps:

- **Define the system's purpose:** Open a live table session, capture guest orders, route tickets, and settle payment.
- **Identify key stakeholders:** Guests, Counter Staff, and Restaurant Owners.
- List functional requirements:
  - Open a table session via QR scan.
  - Capture and submit orders from multiple guests.
  - Route order tickets to the counter dashboard.
  - Track order status and settle payment.
- List non-functional requirements:
  - No login or install required.
  - Real-time data processing.
  - User-friendly interface.

Task 2: Use Case Diagram with Scenarios

31. Objective: Create a Use Case Diagram to represent the system's functionality.
32. Steps:
    - **Identify actors:** Guest, Counter Staff, Payment Gateway.
    - Define use cases:
      - Scan Table QR.
      - Browse Menu.
      - Place Order.
      - Track Order Status.
      - Settle Payment.
    - Create scenarios for each use case:
      - **Example:** A guest scans the table QR, and the system opens a live session showing the menu.
33. Deliverable: Use Case Diagram with detailed scenarios.

```
+-----------+          +-------------------+
```



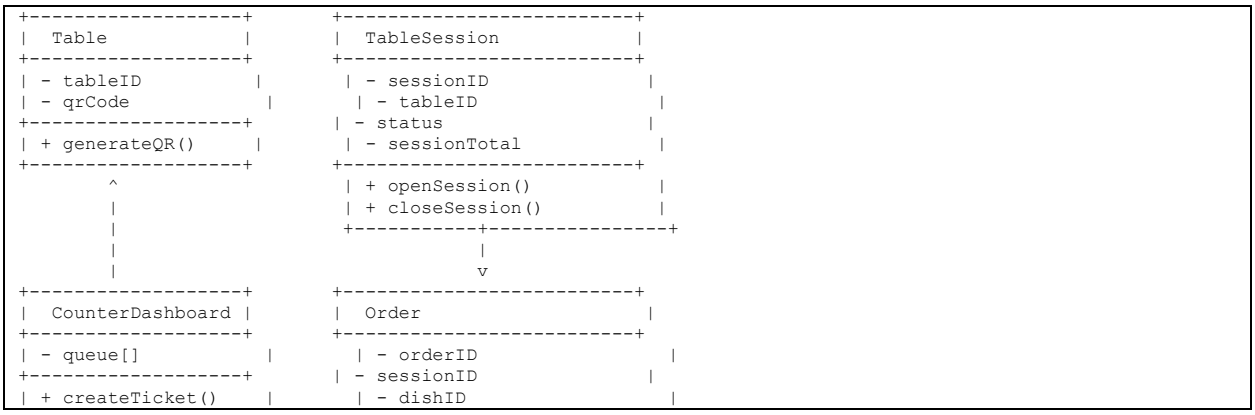
Task 3: Class Diagram

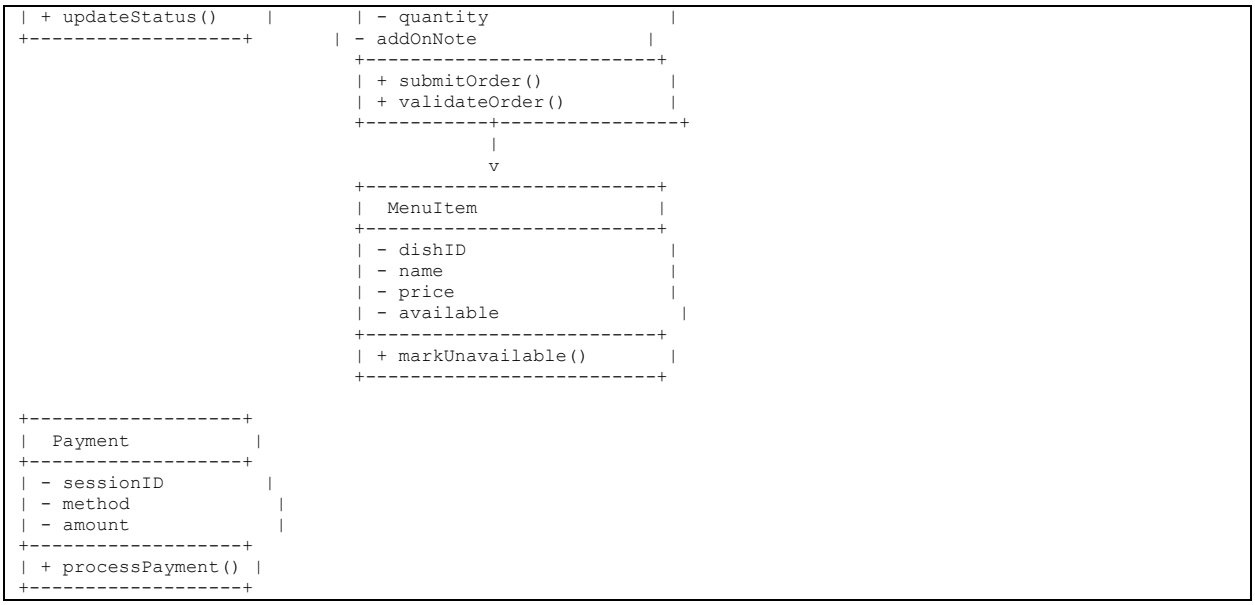
34. Objective: Design the system's structure using a Class Diagram.

35. Steps:

- Identify classes:
  - Table
  - TableSession
  - MenuItem
  - Order
  - CounterDashboard
  - Payment
- Define attributes and methods for each class.
- Establish relationships between classes (e.g., composition, association).

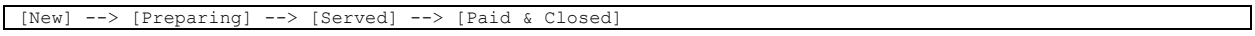
36. Deliverable: Class Diagram with detailed attributes and methods.





Task 4: State Diagram

37. Objective: Model the system's behavior using a State Diagram.
38. Steps:
    - Identify states:
        - New.
        - Preparing.
        - Served.
        - Paid & Closed.
    - Define transitions between states based on events (e.g., "order submitted" triggers "New").
39. Deliverable: State Diagram with states and transitions.

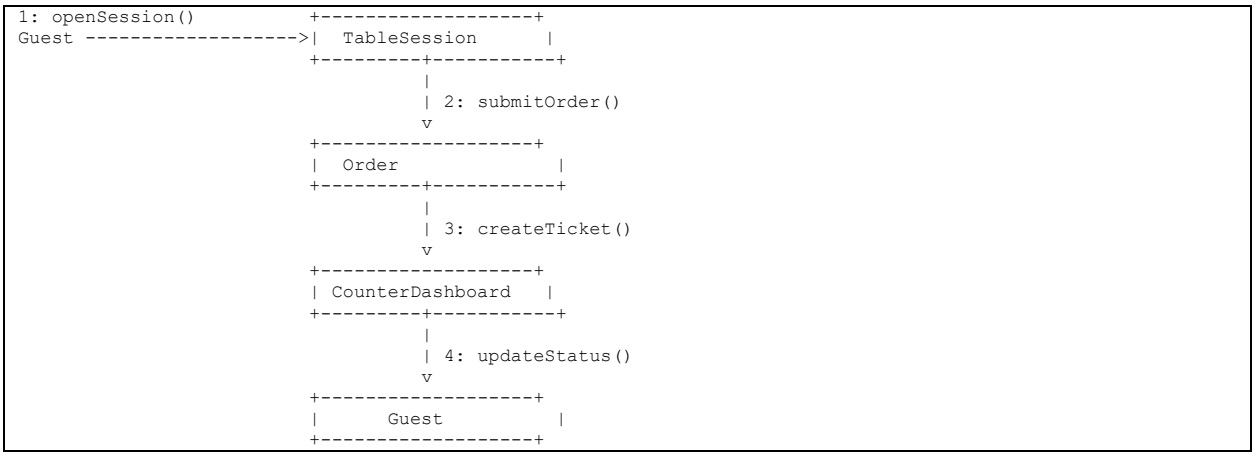


Task 5: Collaboration Diagram

40. Objective: Represent the interaction between objects using a Collaboration Diagram.
41. Steps:

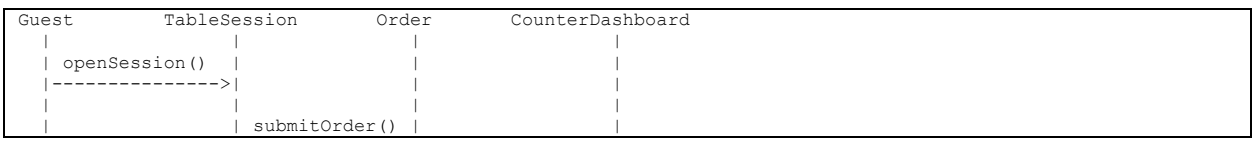
- **Identify objects:** Guest, TableSession, Order, CounterDashboard.
- Define interactions:
  - Guest opens a session.
  - Order is submitted against the session.
  - Counter Dashboard receives the ticket.

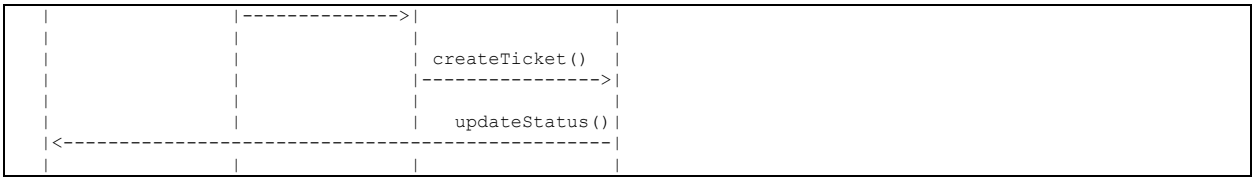
42. Deliverable: Collaboration Diagram showing object interactions.



Task 6: Sequence Diagram

43. Objective: Model the sequence of interactions between objects using a Sequence Diagram.
44. Steps:
    - **Identify objects:** Guest, TableSession, Order, CounterDashboard.
    - Define the sequence of messages:
      - Guest → TableSession: Scan table QR / open session.
      - TableSession → Order: Attach submitted order.
      - Order → CounterDashboard: Create ticket.
45. Deliverable: Sequence Diagram with message sequences.





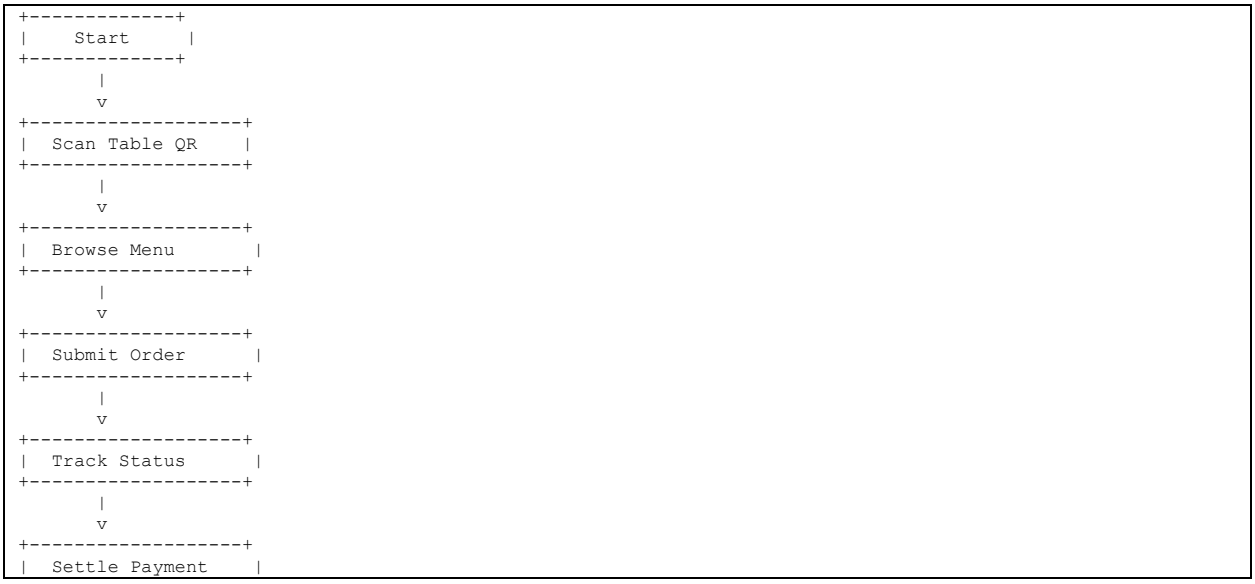
Task 7: Activity Diagram

46. Objective: Represent the system's workflow using an Activity Diagram.

47. Steps:

- Identify activities:
  - Start.
  - Scan Table QR.
  - Browse Menu.
  - Submit Order.
  - Track Status.
  - Settle Payment.
  - End.
- Define decision points (e.g., "Is the table session already open?").

48. Deliverable: Activity Diagram with workflows and decision points.



Expected Outcomes

49. A comprehensive Dinora system design using UML diagrams.
50. A functional prototype of the QR-triggered ordering flow.
51. Documentation of the system's design and implementation.

Evaluation Criteria

52. Completeness: All diagrams and scenarios are included.
53. Accuracy: Diagrams correctly represent the system's structure and behavior.
54. Creativity: Innovative use of the session-based ordering model.
55. Presentation: Clear and professional documentation.

PRACTICAL 8

AIM

Defining Coding Standards and Walkthrough.

Prerequisites

Before starting, ensure you have the following:

- **Programming Language:** JavaScript/TypeScript with React (preferred) or any language of choice.
- Libraries/Frameworks:
  - Front-End: React, Tailwind CSS
  - State/Session Handling: React Context or a lightweight store
  - Build Tool: Vite
  - Deployment (current): Vercel
- **Development Environment:** IDE (e.g., VS Code) or an online editor.
- **Dataset:** Sample menu data (can be synthetic or real-world dish data).
- **Version Control:** Git for code management.

Coding Standards

To ensure maintainability, readability, and scalability, adhere to the following coding standards:

General Guidelines

- Use meaningful variable and function names (e.g., calculateSessionTotal instead of calc).
- Follow a consistent style guide (e.g., Airbnb JavaScript/React style guide, or the equivalent for other languages).
- Write modular code with reusable components and hooks.
- Include comments and docstrings for all functions and components.
- Use version control (Git) and commit messages that describe changes clearly.

File Structure

Organize your project as follows:

```
dinora_web_app/
├── src/
│   ├── components/    # Reusable UI components (MenuItem, OrderTicket)
│   ├── pages/         # Guest menu, Counter dashboard pages
│   ├── hooks/         # Session and order-status hooks
│   ├── utils/         # Utility functions (e.g., session ID generation)
│   └── App.jsx        # Main application entry
├── tests/             # Unit and integration tests
├── package.json       # List of dependencies
└── README.md          # Project documentation
```

Error Handling

- Use try-catch blocks to handle exceptions gracefully, especially around session and payment calls.
- Log errors for debugging (e.g., using a lightweight logging utility).

Testing

- Write unit tests for all functions and components.
- Use testing frameworks like Jest or Vitest.

System Architecture

The system architecture consists of the following components:

56. **QR Table Menu Module:** Handles the QR-triggered browser session (e.g., via scan or manual input).
57. **Live Table Session Module:** Keeps every order for a table tied to one running session.
58. **Counter Dashboard Module:** Surfaces incoming tickets, sorted by table, to the floor staff.
59. **Order Status Module:** Tracks and displays order state to guests and staff.
60. **Payment Module:** Closes out the session on cash, UPI, or card settlement.

Step-by-Step Implementation

Step 1: Set Up the Project

- Create the project directory structure.
- Initialize a Git repository.
- Install required dependencies using npm install.

Step 2: QR Session Handling

- Simulate or capture the table identifier (e.g., from a URL query parameter after the QR scan).
- Example:

```
function getTableIdFromUrl() {
  const params = new URLSearchParams(window.location.search);
  return params.get("table");
}
```

Step 3: Session Creation

- Open a new session or resume an existing one for the scanned table.
- Example:

```
function openOrResumeSession(tableId) {
  // Session lookup / creation logic
  return session;
}
```

Step 4: Order Capture

- Collect dish selections and add-on notes from the guest's browser.
- Example:

```
function submitOrder(sessionId, items) {
  // Attach items to the running session
  return updatedSession;
}
```

Step 5: Counter Dashboard

- Use the incoming order data to populate the counter queue.

- Example:

```
function createTicket(order) {
  if (order.total < minimumThreshold) {
    return "Order below minimum.";
  }
  return "Ticket created for table " + order.tableId;
}
```

Step 6: Reporting Module

- Generate simple visual summaries of orders using a charting library.
- Example:

```
import { BarChart, Bar, XAxis } from "recharts";
// render order counts per table over the session window
```

Testing and Validation

- Write unit tests for each module.
- Validate that a second round of orders lands on the same session rather than a new one.
- Perform integration testing to ensure all modules work together seamlessly.

Conclusion

This lab manual provided a comprehensive guide to developing Dinora, the QR-based dine-in ordering platform. By following the defined coding standards and step-by-step implementation, you can build a robust and scalable system for dine-in ordering.

PRACTICAL 9

AIM

Write the test cases for the identified module.

Objectives

61. Identify the module to be tested.
62. Write comprehensive test cases to validate the module's functionality.
63. Ensure the module meets the system's requirements and performance standards.

Prerequisites

64. Basic understanding of software testing concepts.
65. Knowledge of Dinora's architecture and modules.
66. Familiarity with test case writing techniques.

Tools and Resources

67. Test case management tool (e.g., TestRail, Zephyr, or Excel).
68. Access to the Dinora web app module.
69. Sample menu and session data for testing.
70. Documentation of the module's requirements and specifications.

Lab Tasks

Step 1: Identify the Module

- Select the module to be tested (e.g., Live Table Session Module, Order Capture Module, Counter Dashboard Module, etc.).
- Review the module's requirements and specifications.

Step 2: Define Test Scenarios

- Identify key functionalities of the module.
- Define test scenarios covering normal, edge, and error cases.

Step 3: Write Test Cases

- Write detailed test cases for each scenario.
- Include the following components in each test case:
    - Test Case ID: Unique identifier for the test case.
    - Description: Brief description of the test case.
    - Preconditions: Conditions required before executing the test.
    - Test Steps: Step-by-step instructions to execute the test.
    - Expected Result: Expected outcome of the test.
    - Actual Result: To be filled during test execution.
    - Status: Pass/Fail.

Step 4: Review and Finalize

- Review the test cases for completeness and accuracy.
- Finalize the test cases for execution.

Example Test Cases

Module: Live Table Session Module

Test Case 1: Open a New Table Session

- **Test Case ID:** TC_SESSION_001
- **Description:** Verify that scanning a table's QR code opens a new live session.
- **Preconditions:** No existing open session for the table.
- Test Steps:
    - Scan the QR code fixed to the table.
    - Observe the browser response.
- **Expected Result:** The menu loads instantly in the browser with a new session opened for that table.
- **Actual Result:** [To be filled during execution]
- **Status:** [To be filled during execution]

Test Case 2: Resume an Existing Table Session

- **Test Case ID:** TC_SESSION_002
- **Description:** Verify that a second scan of the same table's QR code resumes the open session rather than creating a new one.
- **Preconditions:** An open session already exists for the table.
- Test Steps:
    - Place an initial order and leave the session open.
    - Scan the same table's QR code again.
- **Expected Result:** The system resumes the same session, showing the existing order history.
- **Actual Result:** [To be filled during execution]
- **Status:** [To be filled during execution]

Test Case 3: Order Submission with Missing Selection

- **Test Case ID:** TC_SESSION_003
- **Description:** Verify that the system handles an empty order submission.
- **Preconditions:** A live session is open for the table.
- Test Steps:
    - Leave the order cart empty.
    - Tap the "Place Order" button.
- **Expected Result:** The system displays a message prompting the guest to select at least one item.
- **Actual Result:** [To be filled during execution]
- **Status:** [To be filled during execution]

Module: Counter Dashboard Module

Test Case 4: New Ticket Appears on the Counter

- **Test Case ID:** TC_COUNTER_001.
- **Description:** Verify that a submitted order appears instantly on the counter dashboard.
- **Preconditions:** The counter dashboard is open and connected.

- Test Steps:
    - Submit an order from a guest's session.
    - Observe the counter dashboard.
- **Expected Result:** The new ticket appears in the "New" column, tagged with the correct table number and items.
- **Actual Result:** [To be filled during execution]
- **Status:** [To be filled during execution]

Test Case 5: Order Status Change Reflects to Guest

- **Test Case ID:** TC_COUNTER_002
- **Description:** Verify that a status change made by staff is reflected on the guest's tracking screen.
- **Preconditions:** An order ticket exists in the "New" state.
- Test Steps:
    - Move the ticket from "New" to "Preparing" on the counter dashboard.
    - Check the guest's order-tracking screen.
- **Expected Result:** The guest's screen updates to show "Preparing" without needing a page refresh.
- **Actual Result:** [To be filled during execution]
- **Status:** [To be filled during execution]

Deliverables

71. A document containing all written test cases.
72. Screenshots or logs of test execution (if applicable).
73. A summary report of the test case review and finalization process.

Conclusion

This lab manual provides a structured approach to writing test cases for Dinora. By following these steps, you can ensure that the identified module is thoroughly tested and meets the system's

requirements. Proper testing will enhance the reliability and performance of the system, ultimately improving the dine-in ordering experience for guests and staff.

PRACTICAL 10

AIM

Demonstrate the use of different Testing Tools with comparison.

Objectives

74. To understand the working principles of QR-triggered table sessions and their significance in dine-in ordering.
75. To explore the integration of testing tools for validating the ordering flow.
76. To evaluate and compare different testing tools (browser-based and automated) used for Dinora.
77. To analyze the performance of the QR-ordering system in real-world floor scenarios.

Theory

1. QR-Triggered Table Sessions

- A table session opens the instant a guest scans the QR code fixed to their table.
- It is commonly used to remove the wait for a server to take the first order.
- Session behavior can vary based on table turnover and party size.

2. Dinora Testing Approach

- Automated tools can simulate QR scans, order submissions, and status changes to validate the flow.
- Manual testing on real devices ensures the browser experience matches guest expectations.

3. Testing Tools

- **Hardware/Manual Tools:** Physical QR codes, guest smartphones, and a counter-side tablet or desktop.
- **Software Tools:** Browser automation frameworks, load-testing tools, and analytics dashboards.

Materials Required

78. Physical or printed QR code for a test table.

79. Guest smartphone with a browser.
80. Counter-side tablet or desktop running the dashboard.
81. Browser automation tool (e.g., Playwright or Selenium).
82. Load-testing tool (e.g., k6 or Apache JMeter).
83. Sample menu dataset (historical or synthetic order data).
84. Notebook for recording observations.

Procedure

Step 1: Setup and Calibration

85. Ensure the QR code links to the correct table session URL.
86. Install the browser automation and load-testing tools on the computer.
87. Connect the counter dashboard device to the same network as the test environment.

Step 2: Data Collection

88. Simulate table sessions from at least 10 test tables using:

- Manual QR scan and order placement
- Browser automation scripts
- Load-testing scripts for concurrent sessions

89. Record the observations in the notebook and sync results with the reporting tool.

Step 3: Automated Testing

90. Run the automation scripts against the ordering flow.
91. Use the tool to generate results (e.g., pass/fail, response time, session integrity).
92. Compare the results with manual testing observations.

Step 4: Comparison of Testing Tools

93. Evaluate the accuracy of each tool by comparing detected issues against manually confirmed issues.
94. Assess the usability of each tool based on ease of setup, scripting effort, and reporting clarity.

95. Measure the efficiency of each tool in terms of test execution speed and coverage.

Step 5: Reporting and Discussion

96. Compile the results in a tabular format for easy comparison.

97. Discuss the strengths and limitations of each tool.

98. Analyze the impact of automated testing on Dinora's ordering-flow reliability.

Observations and Results

Tool	Accuracy (%)	Usability (1-10)	Efficiency (1-10)	Remarks
Manual QR Testing	90%	7	5	Realistic guest experience, but slow and hard to scale.
Playwright/Selenium	94%	7	9	Great for repeatable ordering-flow checks; scripting effort upfront.
k6/JMeter (Load Test)	92%	6	9	Best for testing concurrent table sessions under peak load.
Counter Dashboard Logs	97%	9	8	High visibility into real ticket routing and status accuracy.
Browser DevTools	88%	8	7	Useful for diagnosing slow menu loads on guest devices.

Conclusion

Summarize the findings of the experiment, highlighting the effectiveness of automated and manual testing for the QR-based dine-in ordering flow and the comparative performance of different testing tools.

Precautions

99. Ensure test QR codes are clearly separated from any live/production table codes.

100. Follow the tool vendor's instructions for setup and scripting.

101. Obtain consent from any real participants before collecting test data.

102. Handle guest and payment test data in compliance with privacy practices.

PRACTICAL 11

AIM

Define security and quality aspects of the identified module.

Objective

103. To identify potential security vulnerabilities in the Dinora ordering platform.

104. To establish quality assurance measures for the system's functionality, accuracy, and reliability.

105. To ensure compliance with data protection practices for guest and payment information.

106. To implement best practices for secure session handling and system performance.

Prerequisites

107. Basic understanding of web application concepts.

108. Familiarity with payment and guest-data security standards.

109. Knowledge of software development lifecycle (SDLC).

110. Access to a sample menu and session dataset (simulated or anonymized).

Materials Required

111. Laptop/PC with Node.js installed.

112. Front-end libraries (e.g., React, Tailwind CSS).

113. Session/data store (e.g., a lightweight database or in-memory store).

114. Security testing tools (e.g., OWASP ZAP, Burp Suite).

115. Data encryption tools (e.g., AES, TLS/HTTPS).

116. Quality assurance tools (e.g., Playwright, Jest).

Procedure

Step 1: Identify the Module

117. Define the module to be evaluated (e.g., Live Table Session Module, Payment Module, Counter Dashboard Module).

118. Document the module's functionality, inputs, outputs, and dependencies.

Step 2: Security Aspects

119. Data Security:
    - Ensure all session and payment data is encrypted during transmission (e.g., HTTPS, TLS).
    - Implement encryption for sensitive data at rest (e.g., AES-256).
    - Use secure session identifiers that cannot be easily guessed or reused across tables.

120. Access Control:
    - Define role-based access control (RBAC) for users (e.g., guests, counter staff, owners).
    - Restrict access to the counter dashboard and settlement functions based on staff role.

121. Vulnerability Assessment:
    - Perform penetration testing using tools like OWASP ZAP.
    - Identify and mitigate common vulnerabilities (e.g., session hijacking, XSS, insecure direct object references on table/session IDs).

122. Compliance:
    - Ensure compliance with applicable payment and data protection practices.
    - Conduct regular audits to verify compliance.

Step 3: Quality Aspects

123. Functional Testing:
    - Verify that the module performs its intended functions accurately.
    - Test edge cases (e.g., duplicate QR scans, orders submitted after payment).

124. Performance Testing:
    - Evaluate the system's response time under different loads.
    - Ensure the menu and order ticket load within acceptable time limits on guest devices.

125. Accuracy Testing:
    - Validate that order tickets match exactly what the guest submitted.
    - Calculate reliability metrics (e.g., ticket delivery success rate).
126. Usability Testing:
    - Ensure the guest ordering flow is intuitive and accessible on a phone browser.
    - Gather feedback from end-users (e.g., guests, counter staff).

Step 4: Documentation and Reporting

127. Document all security and quality measures implemented.
128. Create a report summarizing the findings, including:
    - Identified vulnerabilities and their mitigation strategies.
    - Quality metrics (e.g., ticket accuracy, response time).
    - Recommendations for improvement.

Expected Outcomes

129. A secure and reliable Dinora ordering module.
130. Documentation of security and quality measures.
131. A report highlighting vulnerabilities, quality metrics, and recommendations.